C:\Users\mthob\OneDrive\Documents\NetBeansProjects\FederatedIDSArtefact\src\main\java\com\registration\app\federatedid

```java
package com.registration.app.federatedidsartefact.ui;

import javax.swing.*;
import javax.swing.table.DefaultTableCellRenderer;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Random;
import java.io.BufferedReader;
import java.io.File;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.util.Arrays;
import java.util.LinkedHashMap;
import java.util.Map;

public class MainFrame extends JFrame
{

        private static final int MAX_VISIBLE_UI_NODES = 500;

        private static final String READY = "READY";
        private static final String TRAINING = "TRAINING";
        private static final String DONE = "DONE";
        private static final String ERROR = "ERROR";

        private File selectedDatasetFile = null;

        private final JButton generatePartitionsBtn = new JButton("Generate Full Partitions");
        private final JTextArea fullPartitionArea = new JTextArea(12, 30);

        private final List<String[]> fullDatasetRows = new ArrayList<>();
        private String[] fullDatasetHeaders = new String[0];

        private final Map<String, List<String[]>> generatedPartitions = new LinkedHashMap<>();

        private final JTextArea log = new JTextArea();
        private final JProgressBar progress = new JProgressBar(0, 100);

        private final JLabel previewRowsLabel = new JLabel("Preview rows: 0");
        private final JLabel totalColumnsLabel = new JLabel("Columns: 0");
        private final JLabel selectedLabelColumnLabel = new JLabel("Label column: none");

        private final JComboBox<String> labelColumnCombo = new JComboBox<>();
        private final JTextArea labelPreviewArea = new JTextArea(6, 30);

        private final JTextField clientsField = new JTextField("30");
        private final JTextField roundsField = new JTextField("15");

        private final JTextField benignValueField = new JTextField("BENIGN", 12);
        private final JTextField attackValueField = new JTextField("ATTACK", 12);

        private final JComboBox<String> partitionTypeCombo = new JComboBox<>(new String[]
                {
                "IID", "Non-IID"
                });
        private final JSpinner clientPreviewSpinner = new JSpinner(new SpinnerNumberModel(5, 1, 1000, 1));

        private final JTextArea partitionPreviewArea = new JTextArea(10, 30);

        private int previewedRowCount = 0;

        private JButton startBtn;
        private JButton stopBtn;

        private final DefaultTableModel csvModel = new DefaultTableModel();
        private final JTable csvTable = new JTable(csvModel);
        private final JLabel csvPathLabel = new JLabel("No dataset selected");

```

```
72          private final DefaultTableModel nodesModel
73                  = new DefaultTableModel(new Object[]
74                  {
75                  "Node", "Status"
76                  }, 0)
77          {
78                  @Override
79                  public boolean isCellEditable(int r, int c)
80                  {
81                          return false;
82                  }
83          };
84
85          private final JTable nodesTable = new JTable(nodesModel);
86
87          private final JLabel totalClientsLabel = new JLabel("Total clients: 0");
88          private final JLabel visibleClientsLabel = new JLabel("Visible clients: 0");
89          private final JLabel trainingCountLabel = new JLabel("Training: 0");
90          private final JLabel doneCountLabel = new JLabel("Done: 0");
91          private final JLabel errorCountLabel = new JLabel("Errors: 0");
92
93          private TrainingWorker worker;
94
95          private int totalSimulatedClients = 0;
96          private int visibleClients = 0;
97
98          private int trainingCount = 0;
99          private int doneCount = 0;
100         private int errorCount = 0;
101
102         public MainFrame()
103         {
104                 super("Federated IDS Demonstrator (Swing)");
105                 setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
106                 setSize(1200, 760);
107                 setLocationRelativeTo(null);
108
109                 setLayout(new BorderLayout());
110                 add(buildTopControls(), BorderLayout.NORTH);
111                 add(buildTabs(), BorderLayout.CENTER);
112                 add(buildStatusBar(), BorderLayout.SOUTH);
113
114                 log.setEditable(false);
115                 log.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 12));
116         }
117
118         private JComponent buildTopControls()
119         {
120                 JPanel p = new JPanel(new FlowLayout(FlowLayout.LEFT));
121
122                 startBtn = new JButton("Start");
123                 stopBtn = new JButton("Stop");
124                 stopBtn.setEnabled(false);
125
126                 clientsField.setColumns(10);
127                 roundsField.setColumns(6);
128
129                 startBtn.addActionListener(e -> startSimulation());
130                 stopBtn.addActionListener(e -> stopSimulation());
131
132                 p.add(new JLabel("Clients:"));
133                 p.add(clientsField);
134                 p.add(new JLabel("Rounds:"));
135                 p.add(roundsField);
136                 p.add(startBtn);
137                 p.add(stopBtn);
138
139                 return p;
140         }
141
142         private JComponent buildNodesTab()
143         {
144                 JPanel root = new JPanel(new BorderLayout(8, 8));
```

```
145
146                 nodesTable.setFillsViewportHeight(true);
147                 nodesTable.getColumnModel().getColumn(1).setCellRenderer(new StatusRenderer());
148
149                 JButton apply = new JButton("Apply Clients -> Nodes Table");
150                 apply.addActionListener(e ->
151                         {
152                         int requested = parseIntSafe(clientsField.getText(), 30);
153                         initialiseNodeView(requested);
154                         append("[INFO] Applied client count to dashboard.\n");
155                         });
156
157                 JPanel top = new JPanel(new BorderLayout());
158
159                 JPanel left = new JPanel(new FlowLayout(FlowLayout.LEFT));
160                 left.add(new JLabel("UI shows only first " + MAX_VISIBLE_UI_NODES + " nodes to stay responsive.")
161                 left.add(apply);
162
163                 JPanel right = new JPanel(new GridLayout(2, 3, 10, 4));
164                 right.add(totalClientsLabel);
165                 right.add(visibleClientsLabel);
166                 right.add(trainingCountLabel);
167                 right.add(doneCountLabel);
168                 right.add(errorCountLabel);
169
170                 top.add(left, BorderLayout.WEST);
171                 top.add(right, BorderLayout.EAST);
172
173                 root.add(top, BorderLayout.NORTH);
174                 root.add(new JScrollPane(nodesTable), BorderLayout.CENTER);
175
176                 initialiseNodeView(parseIntSafe(clientsField.getText(), 30));
177
178                 return root;
179         }
180
181         private void generateIIDPartitions(int clientCount, List<String[]> benignRows,
182                 List<String[]> attackRows, List<String[]> otherRows)
183         {
184                 List<String[]> allRows = new ArrayList<>();
185                 allRows.addAll(benignRows);
186                 allRows.addAll(attackRows);
187                 allRows.addAll(otherRows);
188
189                 Collections.shuffle(allRows, new Random());
190
191                 for (int i = 1; i <= clientCount; i++)
192                         {
193                         generatedPartitions.put("Client-" + i, new ArrayList<>());
194                         }
195
196                 for (int i = 0; i < allRows.size(); i++)
197                         {
198                         String clientId = "Client-" + ((i % clientCount) + 1);
199                         generatedPartitions.get(clientId).add(allRows.get(i));
200                         }
201         }
202
203         private int findHeaderIndex(String columnName)
204         {
205                 for (int i = 0; i < fullDatasetHeaders.length; i++)
206                         {
207                         if (fullDatasetHeaders[i].trim().equals(columnName))
208                                 {
209                                 return i;
210                                 }
211                         }
212                 return -1;
213         }
214
215         private void generateNonIIDPartitions(int clientCount, List<String[]> benignRows,
216                 List<String[]> attackRows, List<String[]> otherRows)
217         {
```

```
218                 for (int i = 1; i <= clientCount; i++)
219                         {
220                         generatedPartitions.put("Client-" + i, new ArrayList<>());
221                         }
222 
223                 int half = Math.max(1, clientCount / 2);
224 
225                 int benignIndex = 0;
226                 int attackIndex = 0;
227                 int otherIndex = 0;
228 
229                 for (int i = 1; i <= half; i++)
230                         {
231                         String clientId = "Client-" + i;
232                         List<String[]> partition = generatedPartitions.get(clientId);
233 
234                         int share = Math.max(1, benignRows.size() / half);
235                         for (int j = 0; j < share && benignIndex < benignRows.size(); j++)
236                                 {
237                                 partition.add(benignRows.get(benignIndex++));
238                                 }
239                         }
240 
241                 for (int i = half + 1; i <= clientCount; i++)
242                         {
243                         String clientId = "Client-" + i;
244                         List<String[]> partition = generatedPartitions.get(clientId);
245 
246                         int denom = Math.max(1, clientCount - half);
247                         int share = Math.max(1, attackRows.size() / denom);
248                         for (int j = 0; j < share && attackIndex < attackRows.size(); j++)
249                                 {
250                                 partition.add(attackRows.get(attackIndex++));
251                                 }
252                         }
253 
254                 int clientPointer = 1;
255                 while (benignIndex < benignRows.size())
256                         {
257                         generatedPartitions.get("Client-" + clientPointer).add(benignRows.get(benignIndex++));
258                         clientPointer = (clientPointer % clientCount) + 1;
259                         }
260 
261                 while (attackIndex < attackRows.size())
262                         {
263                         generatedPartitions.get("Client-" + clientPointer).add(attackRows.get(attackIndex++));
264                         clientPointer = (clientPointer % clientCount) + 1;
265                         }
266 
267                 while (otherIndex < otherRows.size())
268                         {
269                         generatedPartitions.get("Client-" + clientPointer).add(otherRows.get(otherIndex++));
270                         clientPointer = (clientPointer % clientCount) + 1;
271                         }
272         }
273 
274         private void renderFullPartitionSummary(String partitionType, int benignCount,
275                 int attackCount, int otherCount)
276         {
277                 StringBuilder sb = new StringBuilder();
278 
279                 sb.append("Full dataset partition generation\n");
280                 sb.append("=======================================\n");
281                 sb.append("Partition type: ").append(partitionType).append("\n");
282                 sb.append("Total loaded rows: ").append(fullDatasetRows.size()).append("\n");
283                 sb.append("Benign rows: ").append(benignCount).append("\n");
284                 sb.append("Attack rows: ").append(attackCount).append("\n");
285                 sb.append("Other rows: ").append(otherCount).append("\n");
286                 sb.append("Clients generated: ").append(generatedPartitions.size()).append("\n\n");
287 
288                 sb.append("Client partition sizes\n");
289                 sb.append("---------------------------------------\n");
290 
```

```
                for (Map.Entry<String, List<String[]>> entry : generatedPartitions.entrySet())
                        {
                        sb.append(entry.getKey())
                                .append(": ")
                                .append(entry.getValue().size())
                                .append(" rows\n");
                        }

                fullPartitionArea.setText(sb.toString());
                fullPartitionArea.setCaretPosition(0);
        }

        private void generateFullDatasetPartitions()
        {
                generatedPartitions.clear();
                fullPartitionArea.setText("");

                if (selectedDatasetFile == null)
                        {
                        fullPartitionArea.setText("No dataset selected.");
                        return;
                        }

                if (fullDatasetRows.isEmpty())
                        {
                        fullPartitionArea.setText("Dataset rows are not loaded.");
                        return;
                        }

                Object selected = labelColumnCombo.getSelectedItem();
                if (selected == null)
                        {
                        fullPartitionArea.setText("No label column selected.");
                        return;
                        }

                String labelColumn = selected.toString();
                int labelColIndex = findHeaderIndex(labelColumn);

                if (labelColIndex < 0)
                        {
                        fullPartitionArea.setText("Could not find selected label column.");
                        return;
                        }

                String benignValue = benignValueField.getText().trim();
                String attackValue = attackValueField.getText().trim();
                String partitionType = partitionTypeCombo.getSelectedItem().toString();
                int clientCount = (Integer) clientPreviewSpinner.getValue();

                List<String[]> benignRows = new ArrayList<>();
                List<String[]> attackRows = new ArrayList<>();
                List<String[]> otherRows = new ArrayList<>();

                for (String[] row : fullDatasetRows)
                        {
                        String value = "";
                        if (labelColIndex < row.length && row[labelColIndex] != null)
                                {
                                value = row[labelColIndex].trim();
                                }

                        if (value.equalsIgnoreCase(benignValue))
                                {
                                benignRows.add(row);
                                } else if (value.equalsIgnoreCase(attackValue))
                                {
                                attackRows.add(row);
                                } else
                                {
                                otherRows.add(row);
                                }
                        }
```

```
364
365                 if ("IID".equalsIgnoreCase(partitionType))
366                         {
367                         generateIIDPartitions(clientCount, benignRows, attackRows, otherRows);
368                         } else
369                         {
370                         generateNonIIDPartitions(clientCount, benignRows, attackRows, otherRows);
371                         }
372
373                 renderFullPartitionSummary(partitionType, benignRows.size(), attackRows.size(), otherRows.size())
374                 append("[INFO] Full dataset partitions generated: " + generatedPartitions.size() + " clients\n");
375         }
376
377         private JComponent buildTabs()
378         {
379                 JTabbedPane tabs = new JTabbedPane();
380
381                 tabs.addTab("Data", buildDataTab());
382                 tabs.addTab("Nodes", buildNodesTab());
383                 tabs.addTab("Training Log", new JScrollPane(log));
384                 tabs.addTab("Results", buildResultsTab());
385
386                 return tabs;
387         }
388
389         private void loadCsvPreview(File file)
390         {
391                 csvModel.setRowCount(0);
392                 csvModel.setColumnCount(0);
393                 labelColumnCombo.removeAllItems();
394                 labelPreviewArea.setText("");
395                 partitionPreviewArea.setText("");
396                 fullPartitionArea.setText("");
397                 previewRowsLabel.setText("Preview rows: 0");
398                 totalColumnsLabel.setText("Columns: 0");
399                 selectedLabelColumnLabel.setText("Label column: none");
400                 previewedRowCount = 0;
401
402                 fullDatasetRows.clear();
403                 fullDatasetHeaders = new String[0];
404                 generatedPartitions.clear();
405
406                 try (BufferedReader br = Files.newBufferedReader(file.toPath(), StandardCharsets.UTF_8))
407                         {
408                         String headerLine = br.readLine();
409
410                         if (headerLine == null)
411                                 {
412                                 append("[ERROR] Selected file is empty.\n");
413                                 return;
414                                 }
415
416                         String[] headers = headerLine.split(",", -1);
417                         fullDatasetHeaders = headers;
418
419                         for (String header : headers)
420                                 {
421                                 String cleanHeader = header.trim();
422                                 csvModel.addColumn(cleanHeader);
423                                 labelColumnCombo.addItem(cleanHeader);
424                                 }
425
426                         int previewCount = 0;
427                         String line;
428
429                         while ((line = br.readLine()) != null)
430                                 {
431                                 String[] parts = line.split(",", -1);
432
433                                 if (parts.length < headers.length)
434                                         {
435                                         parts = Arrays.copyOf(parts, headers.length);
436                                         }
```

```
                                fullDatasetRows.add(parts);

                                if (previewCount < 50)
                                        {
                                        csvModel.addRow(parts);
                                        previewCount++;
                                        }
                                }

                        previewedRowCount = previewCount;
                        previewRowsLabel.setText("Preview rows: " + previewCount);
                        totalColumnsLabel.setText("Columns: " + headers.length);

                        if (labelColumnCombo.getItemCount() > 0)
                                {
                                labelColumnCombo.setSelectedIndex(headers.length - 1);
                                }

                        append("[INFO] CSV preview loaded: " + file.getName() + "\n");
                        append("[INFO] Full dataset rows loaded into memory: " + fullDatasetRows.size() + "\n");

                        updatePartitionPreview();
                        } catch (Exception ex)
                        {
                        append("[ERROR] Failed to load CSV preview: " + ex.getMessage() + "\n");
                        }
        }

        private JComponent buildDataTab()
        {
                JPanel root = new JPanel(new BorderLayout(10, 10));

                JButton chooseBtn = new JButton("Choose CSV...");
                chooseBtn.addActionListener(e ->
                        {
                        JFileChooser chooser = new JFileChooser();
                        int result = chooser.showOpenDialog(this);

                        if (result == JFileChooser.APPROVE_OPTION)
                                {
                                File file = chooser.getSelectedFile();
                                selectedDatasetFile = file;
                                csvPathLabel.setText(file.getAbsolutePath());
                                loadCsvPreview(file);
                                }
                        });

                generatePartitionsBtn.addActionListener(e -> generateFullDatasetPartitions());

                labelColumnCombo.addActionListener(e ->
                        {
                        Object selected = labelColumnCombo.getSelectedItem();
                        if (selected != null)
                                {
                                selectedLabelColumnLabel.setText("Label column: " + selected.toString());
                                previewSelectedLabelValues(selected.toString());
                                updatePartitionPreview();
                                }
                        });

                partitionTypeCombo.addActionListener(e -> updatePartitionPreview());
                benignValueField.addActionListener(e -> updatePartitionPreview());
                attackValueField.addActionListener(e -> updatePartitionPreview());
                clientPreviewSpinner.addChangeListener(e -> updatePartitionPreview());

                JPanel filePanel = new JPanel(new BorderLayout(8, 8));
                filePanel.add(chooseBtn, BorderLayout.WEST);
                filePanel.add(csvPathLabel, BorderLayout.CENTER);

                JPanel summaryPanel = new JPanel(new GridLayout(3, 4, 8, 4));
                summaryPanel.add(previewRowsLabel);
                summaryPanel.add(totalColumnsLabel);
```

```
510                 summaryPanel.add(selectedLabelColumnLabel);
511                 summaryPanel.add(new JLabel("Select label column:"));
512 
513                 summaryPanel.add(new JLabel("Benign value:"));
514                 summaryPanel.add(benignValueField);
515                 summaryPanel.add(new JLabel("Attack value:"));
516                 summaryPanel.add(attackValueField);
517 
518                 summaryPanel.add(new JLabel("Partition type:"));
519                 summaryPanel.add(partitionTypeCombo);
520                 summaryPanel.add(new JLabel("Client preview count:"));
521                 summaryPanel.add(clientPreviewSpinner);
522 
523                 JPanel labelPanel = new JPanel(new BorderLayout(8, 8));
524                 labelPanel.add(summaryPanel, BorderLayout.CENTER);
525                 labelPanel.add(labelColumnCombo, BorderLayout.SOUTH);
526 
527                 JPanel partitionControlPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
528                 partitionControlPanel.add(generatePartitionsBtn);
529 
530                 JPanel top = new JPanel(new BorderLayout(8, 8));
531                 top.add(filePanel, BorderLayout.NORTH);
532                 top.add(labelPanel, BorderLayout.CENTER);
533                 top.add(partitionControlPanel, BorderLayout.SOUTH);
534 
535                 csvTable.setFillsViewportHeight(true);
536                 csvTable.setAutoResizeMode(JTable.AUTO_RESIZE_OFF);
537 
538                 labelPreviewArea.setEditable(false);
539                 labelPreviewArea.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 12));
540                 labelPreviewArea.setBorder(BorderFactory.createTitledBorder("Sample values from selected label co
541 
542                 partitionPreviewArea.setEditable(false);
543                 partitionPreviewArea.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 12));
544                 partitionPreviewArea.setBorder(BorderFactory.createTitledBorder("Partition preview"));
545 
546                 fullPartitionArea.setEditable(false);
547                 fullPartitionArea.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 12));
548                 fullPartitionArea.setBorder(BorderFactory.createTitledBorder("Full dataset partition generation")
549 
550                 JSplitPane rightSplit = new JSplitPane(
551                         JSplitPane.VERTICAL_SPLIT,
552                         new JScrollPane(partitionPreviewArea),
553                         new JScrollPane(fullPartitionArea)
554                 );
555                 rightSplit.setResizeWeight(0.5);
556 
557                 JSplitPane lowerSplit = new JSplitPane(
558                         JSplitPane.HORIZONTAL_SPLIT,
559                         new JScrollPane(labelPreviewArea),
560                         rightSplit
561                 );
562                 lowerSplit.setResizeWeight(0.35);
563 
564                 JSplitPane mainSplit = new JSplitPane(
565                         JSplitPane.VERTICAL_SPLIT,
566                         new JScrollPane(csvTable),
567                         lowerSplit
568                 );
569                 mainSplit.setResizeWeight(0.65);
570 
571                 root.add(top, BorderLayout.NORTH);
572                 root.add(mainSplit, BorderLayout.CENTER);
573 
574                 return root;
575         }
576 
577         private JComponent buildResultsTab()
578         {
579                 JPanel panel = new JPanel(new BorderLayout());
580                 JTextArea summary = new JTextArea();
581                 summary.setEditable(false);
582                 summary.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 14));
```

```
583                 summary.setText(
584                         "Scaling mode enabled.\n\n"
585                         + "This demo now separates:\n"
586                         + "1. Total simulated clients\n"
587                         + "2. Visible dashboard clients\n\n"
588                         + "The backend may simulate thousands of nodes,\n"
589                         + "while the UI displays only a manageable subset."
590                 );
591                 panel.add(new JScrollPane(summary), BorderLayout.CENTER);
592                 return panel;
593         }
594
595         private JComponent buildStatusBar()
596         {
597                 JPanel p = new JPanel(new BorderLayout());
598                 progress.setStringPainted(true);
599                 progress.setValue(0);
600                 progress.setString("Idle");
601                 p.add(progress, BorderLayout.CENTER);
602                 return p;
603         }
604
605         private JComponent placeholder(String msg)
606         {
607                 JTextArea a = new JTextArea(msg);
608                 a.setEditable(false);
609                 a.setBorder(BorderFactory.createEmptyBorder(12, 12, 12, 12));
610                 a.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 14));
611                 return new JScrollPane(a);
612         }
613
614         private void initialiseNodeView(int requestedClients)
615         {
616                 totalSimulatedClients = Math.max(1, requestedClients);
617                 visibleClients = Math.min(totalSimulatedClients, MAX_VISIBLE_UI_NODES);
618
619                 nodesModel.setRowCount(0);
620                 for (int i = 1; i <= visibleClients; i++)
621                         {
622                         nodesModel.addRow(new Object[]
623                                 {
624                                 "Client-" + i, READY
625                                 });
626                         }
627
628                 trainingCount = 0;
629                 doneCount = 0;
630                 errorCount = 0;
631
632                 refreshSummaryLabels();
633         }
634
635         private void refreshSummaryLabels()
636         {
637                 totalClientsLabel.setText("Total clients: " + totalSimulatedClients);
638                 visibleClientsLabel.setText("Visible clients: " + visibleClients);
639                 trainingCountLabel.setText("Training: " + trainingCount);
640                 doneCountLabel.setText("Done: " + doneCount);
641                 errorCountLabel.setText("Errors: " + errorCount);
642         }
643
644         private void startSimulation()
645         {
646                 if (worker != null && !worker.isDone())
647                         {
648                         append("[WARN] Already running.\n");
649                         return;
650                         }
651
652                 int requestedClients = parseIntSafe(clientsField.getText(), 30);
653                 int rounds = Math.max(1, parseIntSafe(roundsField.getText(), 15));
654
655                 initialiseNodeView(requestedClients);
```

```
656
657                 append("[INFO] Starting simulation...\n");
658                 append("[INFO] Total simulated clients: " + totalSimulatedClients + "\n");
659                 append("[INFO] Visible dashboard clients: " + visibleClients + "\n");
660                 append("[INFO] Rounds: " + rounds + "\n");
661
662                 if (totalSimulatedClients > visibleClients)
663                         {
664                         append("[INFO] Dashboard capped to first " + visibleClients + " clients for responsivenes
665                         }
666
667                 startBtn.setEnabled(false);
668                 stopBtn.setEnabled(true);
669                 progress.setValue(0);
670                 progress.setString("Running...");
671
672                 worker = new TrainingWorker(totalSimulatedClients, rounds);
673                 worker.execute();
674         }
675
676         private void stopSimulation()
677         {
678                 if (worker != null && !worker.isDone())
679                         {
680                         worker.cancel(true);
681                         append("[INFO] Stop requested.\n");
682                         }
683         }
684
685         private void setNodeStatus(int zeroBasedIndex, String status)
686         {
687                 if (zeroBasedIndex >= 0 && zeroBasedIndex < nodesModel.getRowCount())
688                         {
689                         nodesModel.setValueAt(status, zeroBasedIndex, 1);
690                         }
691         }
692
693         private void append(String s)
694         {
695                 log.append(s);
696                 log.setCaretPosition(log.getDocument().getLength());
697         }
698
699         private static int parseIntSafe(String s, int fallback)
700         {
701                 try
702                         {
703                         return Integer.parseInt(s.trim().replace(",", ""));
704                         } catch (Exception ex)
705                         {
706                         return fallback;
707                         }
708         }
709
710         private String getClientIdFromIndex(int zeroBasedIndex)
711         {
712                 return "Client-" + (zeroBasedIndex + 1);
713         }
714
715         private int getPartitionSize(String clientId)
716         {
717                 List<String[]> rows = generatedPartitions.get(clientId);
718                 return rows == null ? 0 : rows.size();
719         }
720
721         private String describePartitionType(String clientId)
722         {
723                 List<String[]> rows = generatedPartitions.get(clientId);
724
725                 if (rows == null || rows.isEmpty())
726                         {
727                         return "no data";
728                         }
```

```
729
730                 Object selected = labelColumnCombo.getSelectedItem();
731                 if (selected == null)
732                         {
733                         return "unknown";
734                         }
735
736                 int labelColIndex = findHeaderIndex(selected.toString());
737                 if (labelColIndex < 0)
738                         {
739                         return "unknown";
740                         }
741
742                 String benignValue = benignValueField.getText().trim();
743                 String attackValue = attackValueField.getText().trim();
744
745                 int benign = 0;
746                 int attack = 0;
747                 int other = 0;
748
749                 for (String[] row : rows)
750                         {
751                         String value = "";
752                         if (labelColIndex < row.length && row[labelColIndex] != null)
753                                 {
754                                 value = row[labelColIndex].trim();
755                                 }
756
757                         if (value.equalsIgnoreCase(benignValue))
758                                 {
759                                 benign++;
760                                 } else if (value.equalsIgnoreCase(attackValue))
761                                 {
762                                 attack++;
763                                 } else
764                                 {
765                                 other++;
766                                 }
767                         }
768
769                 if (benign > attack && benign > other)
770                         {
771                         return "mostly benign";
772                         } else if (attack > benign && attack > other)
773                         {
774                         return "mostly attack";
775                         } else
776                         {
777                         return "mixed";
778                         }
779         }
780
781         private boolean hasGeneratedPartitions()
782         {
783                 return generatedPartitions != null && !generatedPartitions.isEmpty();
784         }
785
786         private interface UiEvent
787         {
788         }
789
790         private static final class LogEvent implements UiEvent
791         {
792
793                 final String msg;
794
795                 LogEvent(String m)
796                 {
797                         msg = m;
798                 }
799         }
800
801         private static final class StatusEvent implements UiEvent
```

```
802         {
803
804                 final int row;
805                 final String st;
806
807                 StatusEvent(int r, String s)
808                 {
809                         row = r;
810                         st = s;
811                 }
812         }
813
814         private static final class ProgressEvent implements UiEvent
815         {
816
817                 final String label;
818                 final int pct;
819
820                 ProgressEvent(String label, int pct)
821                 {
822                         this.label = label;
823                         this.pct = pct;
824                 }
825         }
826
827         private static final class SummaryEvent implements UiEvent
828         {
829
830                 final int training;
831                 final int done;
832                 final int errors;
833
834                 SummaryEvent(int training, int done, int errors)
835                 {
836                         this.training = training;
837                         this.done = done;
838                         this.errors = errors;
839                 }
840         }
841
842         private final class TrainingWorker extends SwingWorker<Void, UiEvent>
843         {
844
845                 private final int clients;
846                 private final int rounds;
847                 private final Random rnd = new Random();
848
849                 TrainingWorker(int clients, int rounds)
850                 {
851                         this.clients = clients;
852                         this.rounds = rounds;
853                 }
854
855                 @Override
856                 protected Void doInBackground() throws Exception
857                 {
858                         for (int i = 0; i < visibleClients; i++)
859                                 {
860                                 publish(new StatusEvent(i, READY));
861                                 }
862
863                         double sampleFraction = clients > 500 ? 0.05 : 0.25;
864
865                         for (int r = 1; r <= rounds; r++)
866                                 {
867                                 if (isCancelled())
868                                         {
869                                         break;
870                                         }
871
872                                 publish(new LogEvent("[ROUND " + r + "] selecting clients...\n"));
873
874                                 int k = Math.max(1, (int) Math.round(clients * sampleFraction));
```

```
875                                 List<Integer> idxs = new ArrayList<>(clients);
876                                 for (int i = 0; i < clients; i++)
877                                         {
878                                         idxs.add(i);
879                                         }
880 
881                                 Collections.shuffle(idxs, rnd);
882                                 List<Integer> selected = idxs.subList(0, k);
883 
884                                 int currentTraining = selected.size();
885                                 int currentDone = 0;
886                                 int currentErrors = 0;
887 
888                                 for (int idx : selected)
889                                         {
890                                         if (idx < visibleClients)
891                                                 {
892                                                 publish(new StatusEvent(idx, TRAINING));
893                                                 }
894                                         }
895 
896                                 publish(new SummaryEvent(currentTraining, currentDone, currentErrors));
897                                 publish(new LogEvent("[ROUND " + r + "] local training (" + k + " clients)...\n")
898 
899                                 Thread.sleep(250 + rnd.nextInt(250));
900 
901                                 for (int idx : selected)
902                                         {
903                                         boolean fail = rnd.nextDouble() < 0.02;
904 
905                                         if (fail)
906                                                 {
907                                                 currentErrors++;
908                                                 if (idx < visibleClients)
909                                                         {
910                                                         publish(new StatusEvent(idx, ERROR));
911                                                         }
912                                                 } else
913                                                 {
914                                                 currentDone++;
915                                                 if (idx < visibleClients)
916                                                         {
917                                                         publish(new StatusEvent(idx, DONE));
918                                                         }
919                                                 }
920                                         }
921 
922                                 currentTraining = 0;
923                                 publish(new SummaryEvent(currentTraining, currentDone, currentErrors));
924                                 publish(new LogEvent("[ROUND " + r + "] aggregation complete. errors=" + currentE
925 
926                                 int pct = (int) Math.round((r * 100.0) / rounds);
927                                 publish(new ProgressEvent("Round " + r + "/" + rounds, pct));
928 
929                                 Thread.sleep(200);
930 
931                                 for (int idx : selected)
932                                         {
933                                         if (idx < visibleClients)
934                                                 {
935                                                 publish(new StatusEvent(idx, READY));
936                                                 }
937                                         }
938 
939                                 publish(new SummaryEvent(0, 0, currentErrors));
940                                 }
941 
942                         return null;
943                 }
944 
945                 @Override
946                 protected void process(List<UiEvent> chunks)
947                 {
```

```
                        for (UiEvent e : chunks)
                                {
                                if (e instanceof LogEvent le)
                                        {
                                        append(le.msg);
                                        } else if (e instanceof StatusEvent se)
                                        {
                                        setNodeStatus(se.row, se.st);
                                        } else if (e instanceof ProgressEvent pe)
                                        {
                                        progress.setValue(pe.pct);
                                        progress.setString(pe.label);
                                        } else if (e instanceof SummaryEvent sm)
                                        {
                                        trainingCount = sm.training;
                                        doneCount = sm.done;
                                        errorCount = sm.errors;
                                        refreshSummaryLabels();
                                        }
                                }
                }

                @Override
                protected void done()
                {
                        startBtn.setEnabled(true);
                        stopBtn.setEnabled(false);

                        if (isCancelled())
                                {
                                progress.setValue(0);
                                progress.setString("Cancelled");
                                append("[INFO] Simulation cancelled.\n");
                                } else
                                {
                                progress.setValue(100);
                                progress.setString("Done");
                                append("[INFO] Simulation complete.\n");
                                }
                }
        }

        private static class StatusRenderer extends DefaultTableCellRenderer
        {

                @Override
                public Component getTableCellRendererComponent(JTable table, Object value,
                        boolean isSelected, boolean hasFocus,
                        int row, int column)
                {
                        Component c = super.getTableCellRendererComponent(table, value, isSelected, hasFocus, row

                        if (!isSelected)
                                {
                                String s = String.valueOf(value);
                                switch (s)
                                        {
                                        case "READY" ->
                                                c.setBackground(new Color(220, 240, 255));
                                        case "TRAINING" ->
                                                c.setBackground(new Color(255, 245, 200));
                                        case "DONE" ->
                                                c.setBackground(new Color(215, 255, 215));
                                        case "ERROR" ->
                                                c.setBackground(new Color(255, 215, 215));
                                        default ->
                                                c.setBackground(Color.WHITE);
                                        }
                                }

                        return c;
                }
        }
```

```
1021
1022        private void updatePartitionPreview()
1023        {
1024                Object selected = labelColumnCombo.getSelectedItem();
1025
1026                if (selected == null || csvModel.getRowCount() == 0)
1027                        {
1028                        partitionPreviewArea.setText("No dataset loaded yet.");
1029                        return;
1030                        }
1031
1032                String labelColumn = selected.toString();
1033                int labelColIndex = csvModel.findColumn(labelColumn);
1034
1035                if (labelColIndex < 0)
1036                        {
1037                        partitionPreviewArea.setText("Could not find label column.");
1038                        return;
1039                        }
1040
1041                String benignValue = benignValueField.getText().trim();
1042                String attackValue = attackValueField.getText().trim();
1043                String partitionType = partitionTypeCombo.getSelectedItem().toString();
1044                int clientCount = (Integer) clientPreviewSpinner.getValue();
1045
1046                int benignCount = 0;
1047                int attackCount = 0;
1048                int otherCount = 0;
1049
1050                for (int i = 0; i < csvModel.getRowCount(); i++)
1051                        {
1052                        Object valueObj = csvModel.getValueAt(i, labelColIndex);
1053                        String value = valueObj == null ? "" : valueObj.toString().trim();
1054
1055                        if (value.equalsIgnoreCase(benignValue))
1056                                {
1057                                benignCount++;
1058                                } else if (value.equalsIgnoreCase(attackValue))
1059                                {
1060                                attackCount++;
1061                                } else
1062                                {
1063                                otherCount++;
1064                                }
1065                        }
1066
1067                StringBuilder sb = new StringBuilder();
1068                sb.append("Partition preview\n");
1069                sb.append("============================\n");
1070                sb.append("Partition type: ").append(partitionType).append("\n");
1071                sb.append("Preview rows loaded: ").append(previewedRowCount).append("\n");
1072                sb.append("Client preview count: ").append(clientCount).append("\n");
1073                sb.append("Label column: ").append(labelColumn).append("\n");
1074                sb.append("Benign value: ").append(benignValue).append("\n");
1075                sb.append("Attack value: ").append(attackValue).append("\n\n");
1076
1077                sb.append("Class counts in preview\n");
1078                sb.append("----------------------------\n");
1079                sb.append("Benign rows: ").append(benignCount).append("\n");
1080                sb.append("Attack rows: ").append(attackCount).append("\n");
1081                sb.append("Other rows: ").append(otherCount).append("\n\n");
1082
1083                sb.append("Rows per client preview\n");
1084                sb.append("----------------------------\n");
1085                appendRowsPerClientPreview(sb, partitionType, benignCount, attackCount, otherCount, clientCount);
1086
1087                partitionPreviewArea.setText(sb.toString());
1088                partitionPreviewArea.setCaretPosition(0);
1089        }
1090
1091        private void appendRowsPerClientPreview(StringBuilder sb, String partitionType,
1092                int benignCount, int attackCount, int otherCount,
1093                int clientCount)
```

```
        {
                if (clientCount <= 0)
                {
                        sb.append("Invalid client count.\n");
                        return;
                }

                if ("IID".equalsIgnoreCase(partitionType))
                {
                        int total = benignCount + attackCount + otherCount;
                        int baseRows = total / clientCount;
                        int remainder = total % clientCount;

                        for (int i = 1; i <= clientCount; i++)
                        {
                                int rows = baseRows + (i <= remainder ? 1 : 0);
                                sb.append("Client-").append(i).append(": approx ").append(rows).append(" rows\n")
                        }
                }
                else
                {
                        int half = Math.max(1, clientCount / 2);

                        for (int i = 1; i <= clientCount; i++)
                        {
                                if (i <= half)
                                {
                                        int rows = Math.max(1, benignCount / half);
                                        sb.append("Client-").append(i)
                                                .append(": mostly benign, approx ").append(rows).append(" rows\n"
                                }
                                else
                                {
                                        int denom = Math.max(1, clientCount - half);
                                        int rows = Math.max(1, attackCount / denom);
                                        sb.append("Client-").append(i)
                                                .append(": mostly attack, approx ").append(rows).append(" rows\n"
                                }
                        }
                }
        }

        private void previewSelectedLabelValues(String columnName)
        {
                int colIndex = csvModel.findColumn(columnName);

                if (colIndex < 0)
                {
                        labelPreviewArea.setText("Could not find selected label column.");
                        return;
                }

                StringBuilder sb = new StringBuilder();
                sb.append("Selected label column: ").append(columnName).append("\n\n");
                sb.append("Sample preview values:\n");

                int rowCount = Math.min(csvModel.getRowCount(), 15);

                for (int i = 0; i < rowCount; i++)
                {
                        Object value = csvModel.getValueAt(i, colIndex);
                        sb.append("Row ").append(i + 1).append(": ").append(value).append("\n");
                }

                labelPreviewArea.setText(sb.toString());
                labelPreviewArea.setCaretPosition(0);
        }

}
```